Reinforcement Learning

Value Iteration and Policy Iteration in a 5×5 GridWorld

NAME1: Jana Ahmed Farouk

ID1:8603

NAME2: Yara Samy Rizk

ID2: 8706

NAME3: Arwa Mohamed Hamed

ID3:8569

1. Problem Setup

We consider a 5×5 GridWorld with the following reward layout:

R1	**1**	**0**	**-1**	**R2**
2	1	0	-1	-2
2	1	0	-1	-2
2	1	0	-1	-2
2	1	0	-1	-2

The agent can move in four directions: Up, Down, Left, Right. Transitions are stochastic, with 70% probability for the intended direction and 10% for each of the others. Collisions with walls result in no movement.

2. Implementation Details

- **Discount factor:** $\gamma = 0.95$
- **Transition model:** as specified in the assignment
- **Initialization:** Each state's value is initialized to its reward
- **Algorithms implemented:**
  - Value Iteration
  - Policy Iteration
- **Visualization:** A Tkinter GUI displays the grid, rewards, values, and policy arrows. Intermediate iterations are animated live.

3. Value Iteration

Value Iteration is an algorithm that computes the optimal value function by repeatedly applying the Bellman update until convergence.

- **Initialization:** Each state's value is initialized to its immediate reward, as required by the assignment.
- **Update Rule:** For each state s, the algorithm evaluates all possible actions a and computes the expected return:

where:

- P(s'|s,a) is the transition probability,
- R(s') is the reward of the next state,
- γ =0.95 is the discount factor,
- V(s') is the current value of the next state.

The state's value is updated as:

$$V(s) \leftarrow \max_{a} Q(s,a)$$

and the policy is updated to select the action that maximizes Q(s,a).

- **Convergence:** The process continues until the maximum change in values across all states is less than a threshold θ . The result is the optimal value function and policy.

4. Policy Iteration

Policy Iteration alternates between evaluating a fixed policy and improving it until convergence.

- **Initialization:** The algorithm begins with a randomly generated policy.
- **Policy Evaluation:** For each state s, the value is computed under the current policy $\boldsymbol{\pi}$ (s):

This step is repeated until the values converge.

- **Policy Improvement:** For each state, the algorithm checks whether a different action yields a higher expected value:
- **Convergence:** If the policy does not change during improvement, the algorithm terminates. Otherwise, evaluation and improvement steps are repeated.

Policy Iteration often converges in fewer iterations than Value Iteration, but both yield the same optimal policy.

5. Results and Analysis

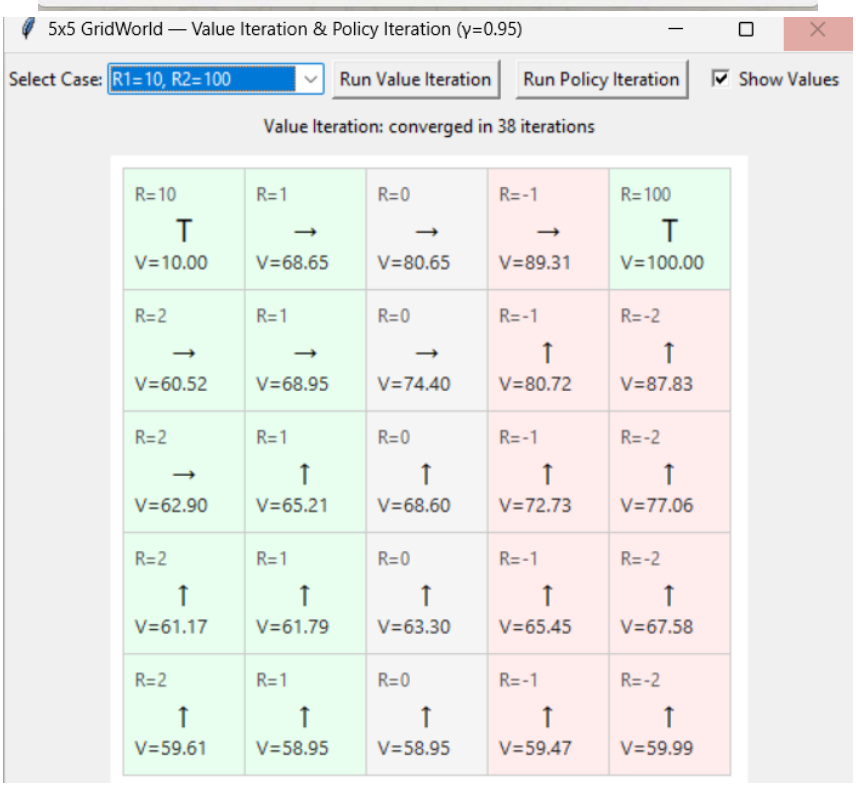
Case 1: R1 = 100, R2 = 110

- Observation: Both R1 and R2 are extremely high. The policy strongly favours reaching either terminal state.
- Policy: Arrows point toward R1 and R2 from all directions.
- Intuition: The agent is highly motivated to reach either reward quickly, while the rest of the grid has minimal influence.

- Run:

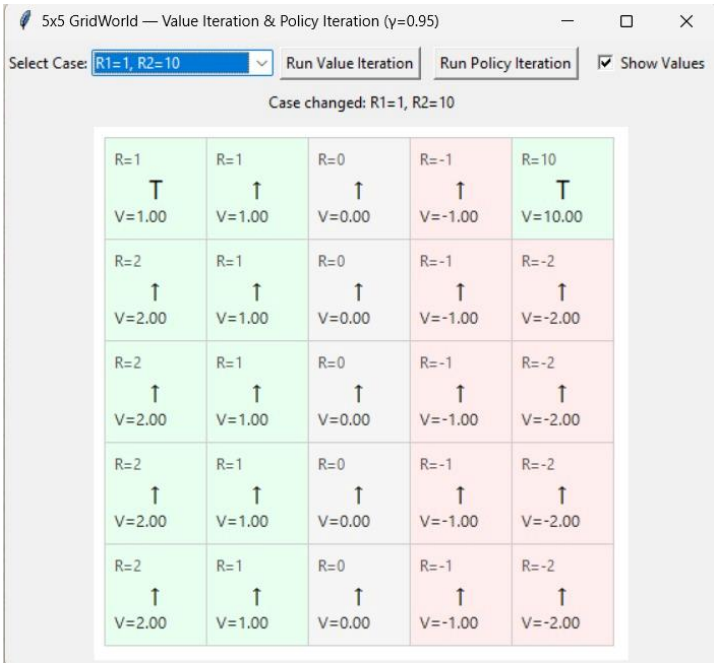
Case 2: R1 = 10, R2 = 100

- **Observation:** R2 dominates. The policy shifts toward the top-right corner.
- **Policy:** Most arrows point toward R2; R1 is less attractive.
- **Intuition:** The agent prefers the higher reward, even if it's farther away.
- Run:



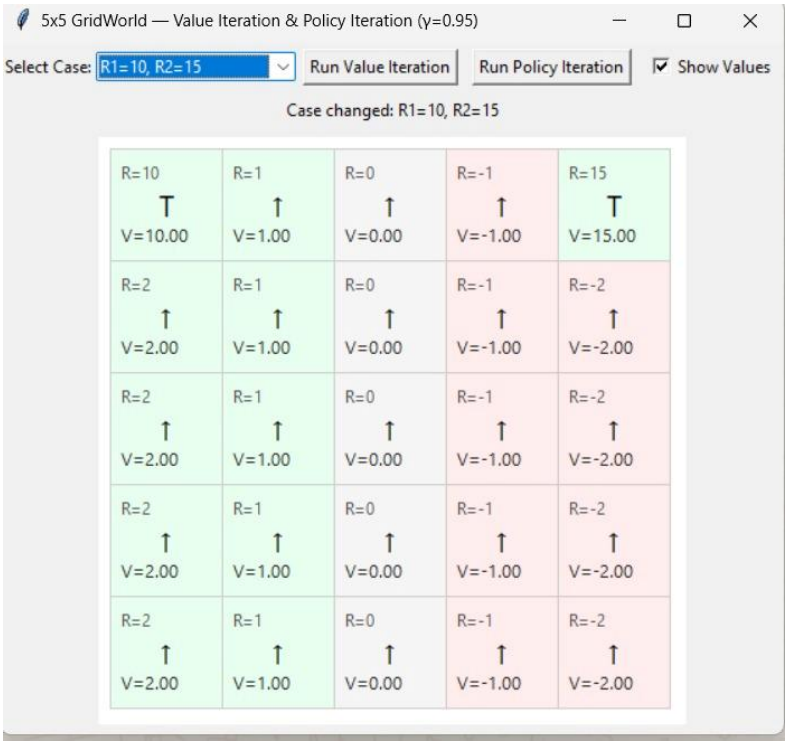
Case 3: R1 = 1, R2 = 10

- Observation: R2 still dominates, but the difference is smaller.
- Policy: Arrows lean toward R2, but some paths toward R1 emerge.
- Intuition: The agent weighs both options more evenly, but R2 still wins due to its higher value.
- Run:



Case 4: R1 = 10, R2 = 15

- **Observation:** The policy is more balanced.
- **Policy:** Arrows split between R1 and R2 depending on proximity.
- **Intuition:** With close reward values, the agent chooses whichever is nearer, optimizing for fewer steps
- Run:


5x5 GridWorld — Value Iteration & Policy Iteration (γ=0.95)
Select Case: R1=10, R2=15
Run Value Iteration
Run Policy Iteration
Show Values
Case changed: R1=10, R2=15
R=10 T V=10.00
R=1 ↑ V=1.00
R=0 ↑ V=0.00
R=-1 ↑ V=-1.00
R=15 T V=15.00
R=2 ↑ V=2.00
R=1 ↑ V=1.00
R=0 ↑ V=0.00
R=-1 ↑ V=-1.00
R=-2 ↑ V=-2.00
R=2 ↑ V=2.00
R=1 ↑ V=1.00
R=0 ↑ V=0.00
R=-1 ↑ V=-1.00
R=-2 ↑ V=-2.00
R=2 ↑ V=2.00
R=1 ↑ V=1.00
R=0 ↑ V=0.00
R=-1 ↑ V=-1.00
R=-2 ↑ V=-2.00
R=2 ↑ V=2.00
R=1 ↑ V=1.00
R=0 ↑ V=0.00
R=-1 ↑ V=-1.00
R=-2 ↑ V=-2.00

5x5 GridWorld — Value Iteration & Policy Iteration (γ=0.95)
Select Case: R1=10, R2=15
Run Value Iteration
Run Policy Iteration
Show Values
Value Iteration: converged in 143 iterations
R=10 T V=10.00
R=1 ↓ V=29.16
R=0 ↓ V=27.45
R=-1 ← V=23.33
R=15 T V=15.00
R=2 ↓ V=33.45
R=1 ↓ V=32.83
R=0 ← V=29.86
R=-1 ← V=25.60
R=-2 ← V=20.51
R=2 ↓ V=36.39
R=1 ← V=34.65
R=0 ← V=31.50
R=-1 ← V=27.13
R=-2 ← V=22.29
R=2 ↓ V=36.79
R=1 ← V=35.15
R=0 ← V=32.06
R=-1 ← V=27.72
R=-2 ← V=22.93
R=2 ← V=36.85
R=1 ← V=35.25
R=0 ← V=32.19
R=-1 ← V=27.89
R=-2 ← V=23.11

6. Policy Iteration

- **Initialization:** Random policy
- **Convergence:** Typically, faster than value iteration (fewer iterations)
- **Result:** Final policies match those from value iteration
- **Intuition:** Policy iteration refines the policy directly, often converging in fewer steps.
- Sample Runs:

Case 1: R1 = 100, R2 = 110

Case 2: R1 = 10, R2 = 100

Case 3: R1 = 1, R2 = 10

Case 4: R1 = 10, R2 = 15

7. GUI Features

- Displays rewards, values, and policy arrows per cell
- Animates each iteration live
- Allows switching between cases
- Shows total iterations taken to converge
- Includes speed control for animation pacing

8. Conclusion

This assignment demonstrates how reward magnitudes influence optimal policies in GridWorld. Value Iteration and Policy Iteration both converge to consistent results, with Policy Iteration often requiring fewer steps. The GUI provides intuitive insight into convergence dynamics and policy evolution.

Let me know if you'd like to add screenshots, references, or export this into a formatted document.